

Introduction to Parallel Algorithms and Programming

Session 1: Introduction to SIMD programming with AVX

Oguz Kaya

Assistant Professor

Université Paris-Saclay and LISN, Gif-sur-Yvette, France

Objectives

- Understand the **SIMD principle**: one instruction, several data elements
- Understand how a SIMD unit works: **load** into vector registers, **operate** on registers, **store** back to memory
- Meet the **AVX** unit of x86 processors and the **intrinsic functions** used to program it
- Learn the **six instructions** needed for today's lab
- Write a first vectorized program: $C = A + B$

Outline

- 1 The SIMD principle
- 2 The AVX unit and its intrinsics
- 3 The six instructions we need
- 4 A first vectorized program: $C = A + B$
- 5 Compiling, running, and the lab assignment

- 1 The SIMD principle
- 2 The AVX unit and its intrinsics
- 3 The six instructions we need
- 4 A first vectorized program: $C = A + B$
- 5 Compiling, running, and the lab assignment

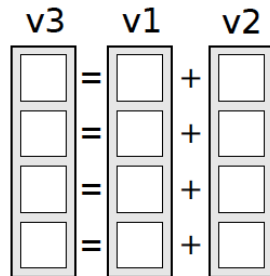
Scalar computation

Principle

A **scalar** processor executes operations one after the other, each operation acting on **one data element** (a scalar).

Example: adding two vectors of 4 elements

```
v3[0] = v1[0] + v2[0];  
v3[1] = v1[1] + v2[1];  
v3[2] = v1[2] + v2[2];  
v3[3] = v1[3] + v2[3];
```



4 instructions, executed sequentially.

Vector computation

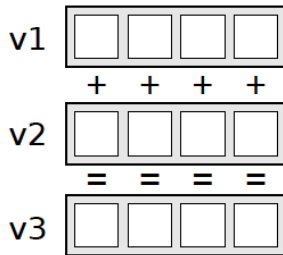
Principle

A **vector** (SIMD) unit executes **one operation on a whole set of data elements** at once: each operation acts on **vectors**.

Example: adding two vectors of 4 elements

$$v3 = v1 + v2;$$

1 instruction, 4 additions performed simultaneously.



Advantages and drawbacks of the vector approach

Advantages

- Each instruction does n times more work, n being the **width** of the vector unit (8 floats for AVX).
- Ideal for algorithms that fit the SIMD paradigm, the same instruction on many data: vector and matrix computations, scientific simulations, image processing, neural networks, ...

Drawbacks

- The gain exists **only if** the algorithm fits the SIMD paradigm; otherwise the vector unit is wasted silicon that could have been cores, cache or registers.
- Programming it by hand is sometimes tedious (compilers can automate part of it, but not reliably).

A bit of history

- 1960s: first project, Solomon
- 1970s: ILLIAC IV (University of Illinois), then vector supercomputers: CDC, Cray, NEC, Hitachi, Fujitsu
- Since 1996: **scalar** consumer processors with a **vector unit** added: Pentium (MMX, SSE, AVX), PowerPC (AltiVec)
- Today: in virtually every processor (x86, ARM, RISC-V), including your phone



Cray-1 (1975), the archetypal vector supercomputer

Vector units in x86 processors

- SSE - Pentium III (1999): 128-bit registers, 4 floats
- SSE2 ... SSE4.2 - Pentium 4 to Core i7
- AVX - Sandy Bridge (2011): **256-bit registers, 8 floats**
- **AVX2** - Haswell (2013): AVX + integer operations + FMA
- AVX-512 - Skylake-X (2017) and later: 512-bit registers, 16 floats (not on all processors)
- Each generation contains the previous one: $\text{SSE}(N) = \text{new instructions} + \text{SSE}(N - 1)$.
- On Linux, `lscpu` lists the versions supported by your processor.

In this course: AVX/AVX2

- Available on every x86 processor sold since ≈ 2013
- 8 floats per instruction
- ARM (Apple M-series, phones) has an equivalent, NEON, with 128-bit registers

- 1 The SIMD principle
- 2 The AVX unit and its intrinsics**
- 3 The six instructions we need
- 4 A first vectorized program: $C = A + B$
- 5 Compiling, running, and the lab assignment

Advanced Vector Extensions (AVX)

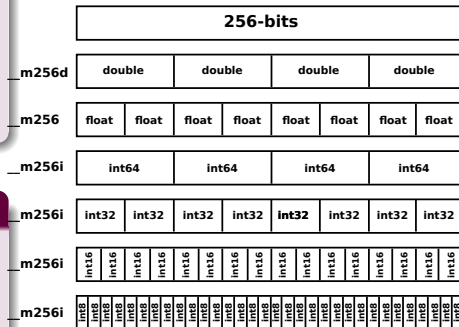
256-bit registers

16 registers of 256 bits on x86-64: YMM0 ... YMM15.

Each one holds 8 floats, or 4 doubles, or 8 32-bit integers, ...

Instructions

- transfers between memory and AVX registers
- arithmetic operations
- logical operations, comparisons
- permutations of the elements



The same 256 bits seen as different data types

How do we program the vector unit?

- **Automatic vectorization**: the compiler tries to vectorize loops by itself (-O3). Works for simple loops, fails silently for many others.
- **Assembly**: direct manipulation of registers and instructions. Low level, painful.
- **Intrinsics**: C functions, one per vector instruction, operating on vector variables. Declared in the header `<immintrin.h>`. **This is what we use.**
- **High-level libraries and directives** (OpenMP simd, C++ SIMD types): hints to the compiler, portability. Later courses.

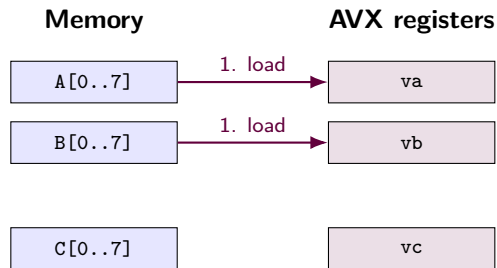
Intrinsics

An intrinsic looks like a function call, but the compiler replaces it by **exactly one vector instruction**: we get the control of assembly with the comfort of C.

General working principle of a SIMD unit

Exactly like scalar registers, except that each operation acts on a whole vector:

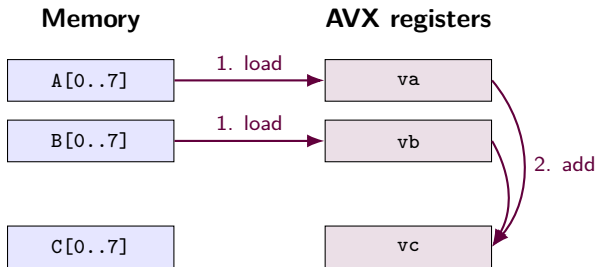
- 1 **Load** data from memory into SIMD registers



General working principle of a SIMD unit

Exactly like scalar registers, except that each operation acts on a whole vector:

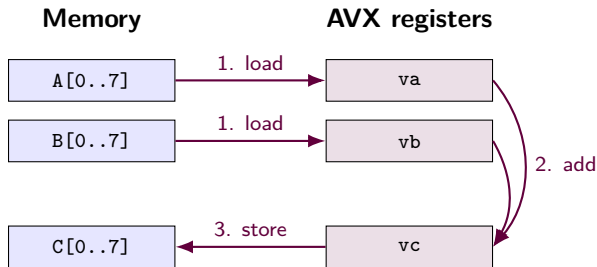
- 1 **Load** data from memory into SIMD registers
- 2 **Operate** on the SIMD registers



General working principle of a SIMD unit

Exactly like scalar registers, except that each operation acts on a whole vector:

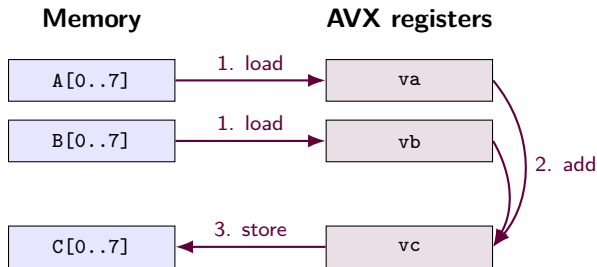
- 1 **Load** data from memory into SIMD registers
- 2 **Operate** on the SIMD registers
- 3 **Store** the result from the registers back to memory



General working principle of a SIMD unit

Exactly like scalar registers, except that each operation acts on a whole vector:

- 1 **Load** data from memory into SIMD registers
- 2 **Operate** on the SIMD registers
- 3 **Store** the result from the registers back to memory



The vector unit **never** operates on memory directly: everything goes through the registers.

- 1 The SIMD principle
- 2 The AVX unit and its intrinsics
- 3 The six instructions we need**
- 4 A first vectorized program: $C = A + B$
- 5 Compiling, running, and the lab assignment

Memory transfers: `loadu` and `storeu`

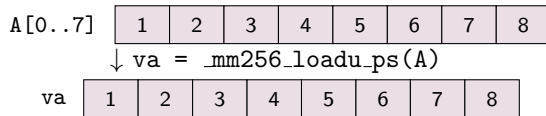
- `_m256` is the AVX2 **float vector register type**: a variable `_m256 x`; holds 8 floats.

Load: 8 consecutive floats into a register

```
_m256 _mm256_loadu_ps(const float *p)  
returns the vector [p[0], p[1], ..., p[7]]
```

Store: a register into 8 consecutive floats

```
void _mm256_storeu_ps(float *p, _m256 v)  
writes p[0] = v0, p[1] = v1, ..., p[7] = v7
```



- `p` is a pointer to the **first** of the 8 elements: `_mm256_loadu_ps(&A[i])` loads `A[i]` to `A[i+7]`.
- The 8 elements must be **contiguous** in memory.

Arithmetic: add, sub, mul, div

All four take two vectors and return a vector; the operation is applied **lane by lane**:

<code>__m256 _mm256_add_ps(__m256 a, __m256 b)</code>	$[a_0 + b_0, a_1 + b_1, \dots, a_7 + b_7]$
<code>__m256 _mm256_sub_ps(__m256 a, __m256 b)</code>	$[a_0 - b_0, a_1 - b_1, \dots, a_7 - b_7]$
<code>__m256 _mm256_mul_ps(__m256 a, __m256 b)</code>	$[a_0 \times b_0, a_1 \times b_1, \dots, a_7 \times b_7]$
<code>__m256 _mm256_div_ps(__m256 a, __m256 b)</code>	$[a_0 / b_0, a_1 / b_1, \dots, a_7 / b_7]$

va	1	2	3	4	5	6	7	8
vb	10	20	30	40	50	60	70	80
vc = <code>_mm256_add_ps(va, vb)</code>								
vc	11	22	33	44	55	66	77	88

- Lane i of the result only depends on lane i of the inputs: **no interaction between lanes**.
- Division is much slower than the other three (as for scalars).

The complete toolbox for today

<code>__m256 x;</code>	declares a vector x of 8 floats
<code>__m256 _mm256_loadu_ps(const float* p)</code>	load 8 floats from memory starting at p
<code>void _mm256_storeu_ps(float* p, __m256 v)</code>	store the 8 floats of v to memory starting at p
<code>__m256 _mm256_add_ps(__m256 a, __m256 b)</code>	element-wise addition
<code>__m256 _mm256_sub_ps(__m256 a, __m256 b)</code>	element-wise subtraction
<code>__m256 _mm256_mul_ps(__m256 a, __m256 b)</code>	element-wise multiplication
<code>__m256 _mm256_div_ps(__m256 a, __m256 b)</code>	element-wise division

Rule of the lab

These six functions are the only intrinsics allowed. Anything else must be built from them: a vector of zeros? prepare `float z[8] = {0,0,0,0,0,0,0,0};` in memory and `loadu` it. The sum of the 8 lanes of a vector? `storeu` it into a `float[8]` and add the 8 values with a scalar loop.

AVX has hundreds of other intrinsics (aligned loads, constants, fused multiply-add, permutations, comparisons, integer/double variants): see the *HPC on Multicore Architectures* course.

- 1 The SIMD principle
- 2 The AVX unit and its intrinsics
- 3 The six instructions we need
- 4 A first vectorized program: $C = A + B$**
- 5 Compiling, running, and the lab assignment

$C = A + B$ on 8 elements: scalar version

- Three arrays of 8 floats
- The loop executes **8 iterations**, each one doing one load of $A[i]$, one load of $B[i]$, one addition, one store to $C[i]$
- Output: 11 22 33 44 55
66 77 88

```
#include <stdio>

int main()
{
    float A[8] = {1, 2, 3, 4, 5, 6, 7, 8};
    float B[8] = {10, 20, 30, 40, 50, 60, 70, 80};
    float C[8];

    // 8 iterations, 1 addition each
    for (int i = 0; i < 8; i++) {
        C[i] = A[i] + B[i];
    }

    for (int i = 0; i < 8; i++) printf("%g ", C[i]);
    printf("\n");
    return 0;
}
```

$C = A + B$ on 8 elements: vectorized version

- Same arrays, same output
- **No loop**: the three steps of the working principle
 - 1 two loads: A, B into va, vb
 - 2 one add: 8 additions at once
 - 3 one store: vc into C
- 4 vector instructions instead of $8 \times 4 = 32$ scalar ones

```
#include <stdio>
#include <immintrin.h>

int main()
{
    float A[8] = {1, 2, 3, 4, 5, 6, 7, 8};
    float B[8] = {10, 20, 30, 40, 50, 60, 70, 80};
    float C[8];

    // 1. load A[0..7] and B[0..7] into registers
    __m256 va = _mm256_loadu_ps(A);
    __m256 vb = _mm256_loadu_ps(B);
    // 2. 8 additions in one instruction
    __m256 vc = _mm256_add_ps(va, vb);
    // 3. store the register into C[0..7]
    _mm256_storeu_ps(C, vc);

    for (int i = 0; i < 8; i++) printf("%g ", C[i]);
    printf("\n");
    return 0;
}
```

$C = A + B$ step by step



$C = A + B$ step by step

A	1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---	---

B	10	20	30	40	50	60	70	80
---	----	----	----	----	----	----	----	----

memory

 $\downarrow$ `va = _mm256_loadu_ps(A)` $\downarrow$ `vb = _mm256_loadu_ps(B)`

va	1	2	3	4	5	6	7	8
----	---	---	---	---	---	---	---	---

vb	10	20	30	40	50	60	70	80
----	----	----	----	----	----	----	----	----

AVX registers

$C = A + B$ step by step

A	1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---	---

B	10	20	30	40	50	60	70	80
---	----	----	----	----	----	----	----	----

memory

 $\downarrow$ `va = _mm256_loadu_ps(A)` $\downarrow$ `vb = _mm256_loadu_ps(B)`

va	1	2	3	4	5	6	7	8
----	---	---	---	---	---	---	---	---

vb	10	20	30	40	50	60	70	80
----	----	----	----	----	----	----	----	----

AVX registers

 $\downarrow$ `vc = _mm256_add_ps(va, vb)`

vc	11	22	33	44	55	66	77	88
----	----	----	----	----	----	----	----	----

$C = A + B$ step by step

A

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

B

10	20	30	40	50	60	70	80
----	----	----	----	----	----	----	----

memory

 $\downarrow$ `va = _mm256_loadu_ps(A)` $\downarrow$ `vb = _mm256_loadu_ps(B)`

va

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

vb

10	20	30	40	50	60	70	80
----	----	----	----	----	----	----	----

AVX registers

 $\downarrow$ `vc = _mm256_add_ps(va, vb)`

vc

11	22	33	44	55	66	77	88
----	----	----	----	----	----	----	----

 $\downarrow$ `_mm256_storeu_ps(C, vc)`

C

11	22	33	44	55	66	77	88
----	----	----	----	----	----	----	----

memory

$C = A + B$ on N elements

- For $N > 8$ we need a loop again, but it advances **8 elements per iteration**: $i = 0, 8, 16, \dots$
- Iteration i handles $A[i..i+7]$, $B[i..i+7]$, $C[i..i+7]$: the pointer passed to `loadu/storeu` is `&A[i]`
- $N/8$ iterations instead of N : ideally **8× faster** (in practice less: memory bandwidth, loop overhead)

```
// C = A + B, N elements (N multiple of 8)
void vecadd_scalar(const float *A, const float *B,
                  float *C, int N)
{
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }
}

void vecadd_avx(const float *A, const float *B,
               float *C, int N)
{
    // one iteration = 8 elements
    for (int i = 0; i < N; i += 8) {
        __m256 va = _mm256_loadu_ps(&A[i]); // A[i..i+7]
        __m256 vb = _mm256_loadu_ps(&B[i]); // B[i..i+7]
        __m256 vc = _mm256_add_ps(va, vb);
        _mm256_storeu_ps(&C[i], vc);        // C[i..i+7]
    }
}
```

$C = A + B$ on N elements

- For $N > 8$ we need a loop again, but it advances **8 elements per iteration**: $i = 0, 8, 16, \dots$
- Iteration i handles $A[i..i+7]$, $B[i..i+7]$, $C[i..i+7]$: the pointer passed to `loadu/storeu` is `&A[i]`
- $N/8$ iterations instead of N : ideally **8× faster** (in practice less: memory bandwidth, loop overhead)
- What if N is not a multiple of 8? Vector loop up to the last multiple of 8, then a **scalar loop** for the remaining $N \bmod 8$ elements (lab exercise).

```
// C = A + B, N elements (N multiple of 8)
void vecadd_scalar(const float *A, const float *B,
                  float *C, int N)
{
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }
}

void vecadd_avx(const float *A, const float *B,
               float *C, int N)
{
    // one iteration = 8 elements
    for (int i = 0; i < N; i += 8) {
        __m256 va = _mm256_loadu_ps(&A[i]); // A[i..i+7]
        __m256 vb = _mm256_loadu_ps(&B[i]); // B[i..i+7]
        __m256 vc = _mm256_add_ps(va, vb);
        _mm256_storeu_ps(&C[i], vc);        // C[i..i+7]
    }
}
```

Summary, and what comes next

- A vector unit applies **one instruction to 8 floats at once**; the potential gain is $8\times$ for AVX.
- Working principle: **load** into registers, **operate** on registers, **store** to memory.
- Intrinsics: one C function per vector instruction, declared in `<immintrin.h>`.
- Six intrinsics are enough to start: `loadu`, `storeu`, `add`, `sub`, `mul`, `div`.

- 1 The SIMD principle
- 2 The AVX unit and its intrinsics
- 3 The six instructions we need
- 4 A first vectorized program: $C = A + B$
- 5 Compiling, running, and the lab assignment

Compiling and running

Compiler options

```
-O3 -fno-tree-vectorize -std=c++11 -mavx2
```

- `-mavx2`: allow the compiler to emit AVX/AVX2 instructions (otherwise the intrinsics do not compile)
- `-fno-tree-vectorize`: forbid the compiler from vectorizing our scalar loops by itself, so that the only vector instructions are the ones we write, and comparisons are fair
- `-O3`: optimize everything else

Lab 1: introduction to SIMD programming

Compiler Explorer (<https://godbolt.org>)

C++, x86-64 gcc, options `-O3 -fno-tree-vectorize -std=c++11 -mavx2`, "Execute the code" ticked in the compiler's Options menu. Save your code locally. Two exercises, scalar vs. vectorized, six intrinsics only; the skeletons provide timing and checking helpers.

Exercise 1: copying an array (copy.cpp)

- Allocate A, B, set $A[i] = i$; scalar copy of A into B
- Vectorized copy: `loadu + storeu` per 8 elements; check $B == A$
- Handle N not a multiple of 8 (scalar remainder loop)
- Time 1000 runs for $N = 1024$ and $N = 2^{20}$: speedup? why the difference?

Exercise 2: inner product $x^T y$ (dot.cpp)

- Allocate x, y, set $x[i] = i$, $y[i] = 1$; scalar inner product
- Vectorized: accumulator of 8 partial sums (`mul`, `add`), `storeu`, add the 8 values; remainder loop
- Same timing comparison as exercise 1
- Is the vectorized result exactly the scalar one? Why?

Contact

Oguz Kaya

Université Paris-Saclay and LISN, Paris, France

oguz.kaya@universite-paris-saclay.fr

www.oguzkaya.com